# Year Two Lecture Notes

BSc Computer Science - Durham University

Matthew Emerson

October 2026

# Contents


## I Artificial Intelligence 1

**1 Module Introduction 1**
- 1.1 Assessment . . . 1
- 1.2 Timetable . . . 1

**2 Machine Learning 1**
- 2.1 Introduction to Machine Learning . . . 1
- 2.2 Machine Learning Workflow and Coding Environment . . . 3
- 2.3 Data Understanding . . . 3
- 2.4 Data Preparation Workflow . . . 4

**3 Artificial Intelligence Search 4**
- 3.1 Coding Real-World Problems for Search Tree Expansion . . . 4
- 3.2 Uninformed Search: BFS, DFS, and Iterative Deepening . . . 4
- 3.3 General Heuristic Search: Greedy Best-First Search and A* Search . . . 4
- 3.4 A* Search and Deriving Iterative Deepening A* Search . . . 4
- 3.5 Heuristic Functions and Their Development . . . 4
- 3.6 Local Search: Hill-Climbing, Simulated Annealing, and Genetic Algorithms . . . 4
- 3.7 Swarm Behaviour, How Ants Forage, and a Simple S-ACO algorithm . . . 4
- 3.8 Applying Different Ant Systems to the TSP . . . 4
- 3.9 General Versions of Particle Swarm Optimisation and Differential Evolution . . . 4
- 3.10 Applying Particle Swarm Optimization to the TSP . . . 4

**4 Ethics and Bias in AI 4**
- 4.1 Introduction to AI Ethics and Related Concepts . . . 4
- 4.2 Societal Impacts of AI . . . 4
- 4.3 Defining Bias and Fairness in ML . . . 4
- 4.4 How to Build Better Systems . . . 4
- 4.5 Value Sensitive Design . . . 4
- 4.6 Evaluations and Audits . . . 4
- 4.7 Bias Mitigation Strategies . . . 4
- 4.8 Transparency and Accountability for Trustworthy Systems . . . 4
- 4.9 Explainable AI . . . 4
- 4.10 Future Trends and Ethical Frontiers . . . 4

## II Data Science 5

**5 Module Introduction 5**
- 5.1 Assessment . . . 5
- 5.2 Timetable . . . 5

**6 Probability and Statistics 5**
- 6.1 Probability and Statistics: Basic Concepts . . . 5
- 6.2 Dependency and Bayes Theorem . . . 8
- 6.3 Discrete Probability Distributions . . . 8
- 6.4 Continuous Probability Distributions . . . 8
- 6.5 Gaussian Distribution and its Characteristics . . . 8
- 6.6 Correlations . . . 8
- 6.7 Central Limit Theorem and Sampling . . . 8
- 6.8 Hypothesis Testing . . . 8

6.9 Data Types and Visualisation . . . 8
6.10 Basic Data Analysis with Python . . . 8

**7 Data Cleaning and Analytics 8**
7.1 Data Quality and How to Produce High-Quality Data . . . 9
7.2 Working with Files . . . 10
7.3 Pandas and Various Features of NumPy . . . 10
7.4 Data Cleaning and Preparation . . . 10
7.5 Python Data Visualisation Libraries . . . 10
7.6 Data Analytics and Reporting . . . 10

**8 Computer Graphics 10**

**9 Image Processing 10**

**III Networks and Systems 11**

**10 Module Introduction 11**
10.1 Assessment . . . 11
10.2 Timetable . . . 11

**11 Cyber Security 11**
11.1 Introduction, Threat Modelling, and Applied Cryptography . . . 11
11.2 System Security . . . 14
11.3 Defence . . . 16

**12 Networks 16**

**13 Databases 16**
13.1 Enhanced Entity-Relationship Model . . . 16
13.2 Semistructured Databases (XML) . . . 16
13.3 XML Data Manipulation (XPath and XQuery) . . . 16
13.4 Transactions . . . 16
13.5 Concurrency Control . . . 16
13.6 Distributed Transactions . . . 16
13.7 Distributed Concurrency Control . . . 16

**14 Distributed Systems 16**
14.1 Introduction to Distributed Systems . . . 17
14.2 Middleware and RMI Technologies . . . 17
14.3 Message-Oriented Middleware and Web Services . . . 17
14.4 Replication Models . . . 17
14.5 Concurrent Events and Consistency Control . . . 17
14.6 Fault Tolerance . . . 17
14.7 Load Distribution . . . 17

**IV Programming Paradigms 18**

**15 Module Introduction 18**
15.1 Assessment . . . 18
15.2 Timetable . . . 18

**16 Systems Programming** **18**
16.1 Version Control and UNIX . . . 18
16.2 The Basics of Shell . . . 19
16.3 The Basics of C . . . 19
16.4 Control Flow and Functions . . . 21
16.5 Data Types and Structures . . . 21
16.6 Advanced Bash Scripting . . . 21
16.7 Pointers . . . 21
16.8 Memory Management . . . 21
16.9 Debugging . . . 21
16.10Scope and Recursion . . . 21
16.11Libraries . . . 21
16.12stdlib . . . 21
16.13C++ . . . 21

**17 Functional Programming** **21**

**18 Object Oriented Programming** **21**
18.1 Java Fundamentals . . . 21
18.2 OOP with Classes and Objects . . . 21
18.3 Encapsulation . . . 21
18.4 Inheritance . . . 21
18.5 Abstraction with Abstract Classes . . . 21
18.6 Abstraction with Interfaces . . . 21
18.7 Polymorphism . . . 21
18.8 Java Collections . . . 21
18.9 Best Practises . . . 21

## V Software Engineering **22**

**19 Module Introduction** **22**
19.1 Assessment . . . 22
19.2 Timetable . . . 22

**20 Project Management** **22**
20.1 Your Team, Formation, and Leadership . . . 22

**21 Software Engineering** **22**

**22 Legal, Social, Ethical, and Professional Issues** **22**
22.1 Introduction to Software Engineering and Software Engineering Ethics . . . 23

**23 Group Project** **23**

## VI Theory of Computation **24**

**24 Module Introduction** **24**
24.1 Assessment . . . 24
24.2 Timetable . . . 24

**25 Models of Computation** **24**
25.1 Finite-State Automata and Regular Languages . . . 24
25.2 Büchi Automata and Omega-Regular Languages . . . 28
25.3 Linear-Time Logic and Translating Into Büchi Automata . . . 28

**26 Algorithms and Complexity** **28**
26.1 Asymptotic Complexity and Notation . . . 28
26.2 Greedy Algorithms . . . 28
26.3 Divide and Conquer Algorithms . . . 29
26.4 Dynamic Programming Algorithms . . . 29
26.5 Machinings in Graphs . . . 29

**VII Appendix** **30**

**27 Artificial Intelligence** **30**
27.1 Module Professors . . . 30
27.2 References . . . 30
27.3 List of Figures . . . 30
27.4 Reading List . . . 30

**28 Data Science** **31**
28.1 Module Professors . . . 31
28.2 References . . . 31
28.3 List of Figures . . . 31
28.4 Reading List . . . 31

**29 Networks and Systems** **31**
29.1 Module Professors . . . 31
29.2 References . . . 32
29.3 List of Figures . . . 32
29.4 Reading List . . . 32

**30 Programming Paradigms** **32**
30.1 Module Professors . . . 32
30.2 References . . . 33
30.3 List of Figures . . . 33
30.4 Reading List . . . 33

**31 Software Engineering** **33**
31.1 Module Professors . . . 33
31.2 References . . . 34
31.3 List of Figures . . . 34
31.4 Reading List . . . 34

**32 Theory of Computation** **34**
32.1 Module Professors . . . 34
32.2 References . . . 35
32.3 List of Figures . . . 35
32.4 Recommended Reading . . . 35

# Part I
# Artificial Intelligence

## 1 Module Introduction

Artificial Intelligence has the module code COMP2261, and is led by Dr Yang Long.

### 1.1 Assessment

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Machine Learning | 9th Oct | 15th Dec | 21st Jan | Dr Yang Long | 50% |
| AI Search | 15th Jan | 18th Feb | 18th Mar | Prof Iain Stewart | 25% |
| Bias in AI | 5th Feb | 30th Mar | 6th May | Dr Shauna Concannon | 25% |

Table 1: Assessments for Artificial Intelligence.

### 1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Monday | 14:00 - 16:00 | Computer Class | TLC025 | - |
| Tuesday | 16:00 - 17:00 | Lecture | W103 | Dr Yang Long |
| Friday | 15:00 - 16:00 | Lecture | W103 | Dr Yang Long |

Table 2: Michaelmas Term Timetable for Artificial Intelligence.

## 2 Machine Learning

This section is taught by Dr Yang Long. The recommended reading for this topic is:

- Mathematics for Machine Learning (Deisenroth, M.P., Faisal, A.A., and Ong, C.S., 2020).
- Mastering Machine Learning with scikit-learn (Hackeling, G., 2017).

### 2.1 Introduction to Machine Learning

#### 2.1.1 Module Information

This module is organised around a direct connection between teaching and practical work. The lectures introduce the concepts, mathematics, assumptions, and experimental consequences needed for that week's practical. The practical then produces the evidence used in the corresponding coursework report. This can be seen in Table 3.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

| Stage | What you do | What you keep |
|---|---|---|
| Lecture | Understand the main question, mechanisms, assumptions, and key equations. | Notes in your own words and questions that still need an answer. |
| Practical | Implement, test, compare, visualise, and discuss the method. | A notebook that runs from top to bottom with outputs included. |
| Weekly Report | Select the strongest evidence and explain why the result occured. | A concise report that links theory, method, result, and interpretation. |
| Portfolio | Revise the nine reports and connect ideas across the module. | A coherent record of technical progress and developing judgement. |

Table 3: The learning and coursework model.

For more information about expectations for evidence, see (Long and Durham University, 2026, pp.3-4).

### 2.1.2 What is Machine Learning?

**Definition 2.1.2.1**: Arthur Samuel said in 1959 that machine learning is the

> "field of study that gives computers the ability to learn without being explicitly programmed."

The field arose from the question – can a computer learn on its own in order to perform a specific task, rather than relying on a programmer explicitly setting up rules for it? Similarly, Tom Mitchell defined machine learning in 1997 as

> "A computer program is said to learn from experience $E$ with respect to some class of tasks $T$ and performance measure $P$, if its performance at tasks in $T$, as measured by $P$, improves with experience $E$."

From this, we can define a well-defined learning task as $< T, P, E >$.

There are three typical types of machine learning, these are:

- **Definition 2.1.2.2**: Supervised learning seeks to learn the mapping (rules) between inputs and outputs. Labelled data (correct answers) is provided in past input and output pairs during the learning process to train the model on how it should behave for previously unseen data.
- **Definition 2.1.2.3**: Unsupervised learning seeks to discover the hidden pattern from a set of inputs. Unlabelled data is provided as the past input during the learning process. The program will then attempt to group similar instances into groups.
- **Definition 2.1.2.4**: Reinforcement learning works by providing occasional positive and negative feedback to reinforce behaviours. Good behaviours are rewarded with treats and therefore become more common whereas bad behaviours are punished and therefore become less common. This is a balance between exploration (of unchared territory) and exploitation (of current knowledge).

### 2.1.3 Differences between AI, ML, and DL

**Definition 2.1.3.1**: Artificial intelligence is defined as human intelligence being exhibited by machines.

**Definition 2.1.3.2**: In simple terms, machine learning can be defined as an approach to achieve artificial intelligence.

**Definition 2.1.3.3**: Deep learning is defined as a technique for achieving machine learning. As such, we can consider AI to be a subset of ML and ML to be a subset of DL.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

## 2.2 Machine Learning Workflow and Coding Environment

### 2.2.1 Machine Learning Workflow

**Definition 2.2.1.1**: To train a model, you use data to create a model and fine-tune it to make better predictions. We can then use this predictive model to make predictions on previously unseen data in order to perform specific tasks.

The machine learning workflow is set out as shown below:

1. **Definition 2.2.1.2**: Project Framing: When setting out to create a model, we must first frame the problem. Here, we set out a clear objective that we want the model to achieve. We then define inputs and outputs and establish success metrics (e.g., prediction accuracy).
2. **Definition 2.2.1.3**: Data Preparation: The next step is to prepare the data. To do that, we identify features (inputs) and a label (output). We then carry out data cleaning on the dataset and split the data into training and test sets.
3. **Definition 2.2.1.4**: Algorithm Selection: Next, we select the algorithm we want to use for the model based on the dataset and the objective.
4. **Definition 2.2.1.5**: Model Training: The training data is used to incrementally improve the model's performance. The progress made in this step is dependant on the quality of the work done in the previous two stages.
5. **Definition 2.2.1.6**: Model Testing: The next step is to take the model and test it against previously unseen data (the test set).
6. **Definition 2.2.1.7**: Hyperparameter Tuning: We can then attempt to improve the training process to further increase the model's accuracy.
7. **Definition 2.2.1.8**: Inference/Prediction: Finally, the model can be deployed for the task outlined in the first step.

### 2.2.2 Coding Environment

When programming, we traditionally set out the features as variable $X$ and the target label as $Y$.

## 2.3 Data Understanding

**Definition 2.3.0.1**: A sample is the unit for which the model recieves features and, during training, a label.

We then set up a feature matrix $X$ where each row is a different sample and each column is a new feature and $y$ is a matrix where each row is sample's label corresponding to the sample's features in $X$.

**Definition 2.3.0.2**: Nearest-centroid classification attempts to classify the samples by setting out one centre for each species and then classifying them based on their nearest centroid. The formula for calculating this is:

$$\mu_c = \left(\frac{1}{N_c}\right) \sum_i : y^{(i)} = cx^{(i)} \tag{1}$$

For each species $c$, average the feature vectors belonging to that species. Then, subtract the centroid from the new example, square the differences, add them, then take the square root, as shown below.

$$d(x, \mu_c) = ||x = \mu_x||_2 \tag{2}$$

To predict a sample's species, compute the distances from each species' centroid and choose the class with the smallest distance.

$$\hat{y} = arg\ min_c||x - \mu_c||_2 \tag{3}$$


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 2.4 Data Preparation Workflow

## 3 Artificial Intelligence Search

This section is taught by Professor Iain Stewart. There is no recommended reading for this topic however there may be an additional video to watch after lectures to enable more content to be covered.

Watch the FAQs video.

### 3.1 Coding Real-World Problems for Search Tree Expansion

Watch ahead of time - this will not be covered in lectures. Week 5 covers no content.

### 3.2 Uninformed Search: BFS, DFS, and Iterative Deepening

Watch ahead of time - this will not be covered in lectures. Week 5 covers no content.

### 3.3 General Heuristic Search: Greedy Best-First Search and A* Search

### 3.4 A* Search and Deriving Iterative Deepening A* Search

### 3.5 Heuristic Functions and Their Development

### 3.6 Local Search: Hill-Climbing, Simulated Annealing, and Genetic Algorithms

### 3.7 Swarm Behaviour, How Ants Forage, and a Simple S-ACO algorithm

### 3.8 Applying Different Ant Systems to the TSP

### 3.9 General Versions of Particle Swarm Optimisation and Differential Evolution

### 3.10 Applying Particle Swarm Optimization to the TSP

## 4 Ethics and Bias in AI

This section is taught by Dr Shauna Concannon. The recommended reading is handled on a weekly basis for this topic.

### 4.1 Introduction to AI Ethics and Related Concepts

### 4.2 Societal Impacts of AI

### 4.3 Defining Bias and Fairness in ML

### 4.4 How to Build Better Systems

### 4.5 Value Sensitive Design

### 4.6 Evaluations and Audits

### 4.7 Bias Mitigation Strategies

### 4.8 Transparency and Accountability for Trustworthy Systems

### 4.9 Explainable AI

### 4.10 Future Trends and Ethical Frontiers


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part II
# Data Science

## 5 Module Introduction

Data Science has the module code COMP2271 and is led by Professor Thomas Erlebach.

### 5.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 4. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Data Cleaning & Analytics | Week 11 | - | 11th Feb | Dr Igor Razgon | 25% |
| Image Processing | 15th Jan | 2nd Mar | 6th May | Dr George Koulieris | 25% |
| Probability and Graphics | May Exam | - | - | Razgon & Koulieris | 50% |

Table 4: Assessments for Data Science.

**Data Cleaning & Analytics Benchtest Information**: The Data Cleaning & Analytics test in Week 11 is a closed book, multiple choice test, and will be held during practicals (mine is Friday 4pm).

- Closed-book assessment over one hour. Check information closer to the time.
- Comprised of 15 multiple-choice questions, split equally between: basic NumPy, basic Pandas, data cleaning, data analytics, and visualisation.

### 5.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 5.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Monday | 12:00 - 13:00 | Lecture | W103 | Dr Igor Razgon |
| Friday | 10:00 - 11:00 | Lecture | W103 | Dr Igor Razgon |
| Friday | 16:00 - 18:00 | Computer Class | CC-0007 | - |

Table 5: Michaelmas Term Timetable for Data Science.

## 6 Probability and Statistics

This section is taught by Dr Igor Razgon. The recommended reading for this topic is:

- Probability and Statistics (Degroot, M.H. and Schervish, M.J., 2012).

### 6.1 Probability and Statistics: Basic Concepts

#### 6.1.1 Counting Theory

**Definition 6.1.1.1**: The product rule states that, if a procedure can be broken down into a sequence of two tasks with $n_1$ ways to do the first task and $n_2$ ways to do the second task, then there are $n_1 n_2$ ways to complete the entire procedure.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Example 6.1.1.1**: John wants to fly to Canada from Germany with a connecting flight through the UK. There are 3 flights from Germany to the UK, and 2 flights from the UK to Canada. How many possible flight combinations are there?

*Solution*: $3 \times 2 = 6$.

□

**Definition 6.1.1.2**: The sum rule states that, if you consider two independent sub-tasks (we only have to do one), if the first can be done in $n_1$ ways, and the second can be done in $n_2$ ways, then the total number of ways to complete the task is $n_1 + n_2$.

**Example 6.1.1.2**: Mary wants to choose a student project from three lists: lecturer 1 offers 5 projects, lecturer 2 offers 2 projects, and lecturer 3 offers 10 projects. How many possible options for her?

*Solution*: $5 + 2 + 10 = 17$.

□

**Definition 6.1.1.3**: The inclusion-exclusion principle states that, if two tasks can be done at the same time (unlike the sum rule in which they are independent), then there is a possibility of overcounting since some ways may be counted twice. To counter this, we add up the number of ways to do each task, then subtract the number of ways to do both tasks.

$$|A \cup B| = (|A| + |B|) - (|A \cap B|) \tag{4}$$

$$|A \cup B \cup C| = (|A| + |B| + |C|) - (|A \cap B| + |A \cap C| + |B \cap C|) + (|A \cap B \cap C|) \tag{5}$$

**Definition 6.1.1.4**: An r-permutation of a set of $r$ distinct objects is one ordered arrangement. We denote the number of r-permutations of a set with $n$ elements in as $P(n, r)$.

$$P(n, r) = n \times (n-1) \times (n-2) \times ... \times (n-r+1) = \frac{n!}{(n-r)!} \tag{6}$$

Suppose that a set consists of $n$ objects, of which $n_1$ are of one type (indistinguishable from one another), $n_2$ are of a second type, ..., and $n_k$ are of a $k_{th}$ type where $n = n_1 + n_2 + ... + n_k$. The number of permutations of the objects can be calculated with (7).

$$P(n; n_1, n_2, ..., n_k) = \frac{n!}{n_1! n_2! ... n_k!} \tag{7}$$

**Definition 6.1.1.5**: A r-combination is a counting of distinct objects in an unordered arrangement of length $r$. The number of $r$ combinations of a set with $n$ distinct elements is given by (8).

$$C(n, r) = \binom{n}{r} = \frac{n!}{(n-r)! \times r!} \tag{8}$$

**Example 6.1.1.3**: Assume elements $A_1, A_2, A_3, A_4$ are in a set. We want to choose 2 elements from them.

*Solution*:

- If the order of elements is important: $P(4, 2) = \frac{4!}{(4-2)!} = 12$.
- If the order of elements is not important: $C(4, 2) = \frac{4!}{(4-2)!2!} = 6$.

□


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 6.1.2 Probability Theory

**Definition 6.1.2.1**: Probability is the formalisation of uncertain events. It enables one to infer probabilities of interest based on assumptions and observations.

**Definition 6.1.2.2**: Statistics is the practice of the science of collecting and analysing data. It deals with experimental data and extracting knowledge from it.

**Definition 6.1.2.3**: An experiment is a procedure that yields one of a given set of possible outcomes.

**Definition 6.1.2.4**: The sample space of the experiment is the set of possible outcomes.

**Definition 6.1.2.5**: An event is any subset of the sample space.

**Theorem 6.1.2.1**: The Theory of Probability states that, given a finite sample space of all outcomes $S$, assuming that all outcomes are equally likely, for an event set $E$ where $E \subseteq S$, then the probability of occurrence of the event is given by (9).

$$P(E) = \frac{|E|}{|S|} \tag{9}$$

*Proof*: Proof omitted.

□

**Definition 6.1.2.6**: The axioms of probability are as follows:

- $P(E)$ is always between 0 and 1: $0 \leq P(E) \leq 1$.
- $P(S) = 1$.
- For disjoint events $E_1, E_2, ..., E_n$ (with no intersection), $P(E_1 \cup E_2 \cup ... \cup E_n) = \sum_{i=1}^{n} P(E_i)$.

Then, from the Axioms of Probability above, we can derive the following:

- **Definition 6.1.2.7**: The complement rule states that $P(\overline{E}) = P(S - E) = P(S) - P(E) = 1 - P(E)$.
- **Definition 6.1.2.8**: The impossible rule states that $P(\emptyset) = 0$.
- **Definition 6.1.2.9**: The general addition rule states that $P(E_1 \cup E_2) = P(E_1) + P(E_2) - P(E_1 \cap E_2)$.
- If $E_1 \subseteq E_2$, then $P(E_1) \leq P(E_2)$.

**Example 6.1.2.1**: A sequence of 10 bits is randomly generated. What is the probability that at least 1 of these is 0?
*Solution*: Let $E$ be an event in which all bits are 1.

$$P(E) = \frac{|E|}{|S|} = \frac{1}{2^{10}}$$

Then, by the complement rule, $P(\overline{E}) = P(1 - E)$ where $\overline{E}$ is the event in which at least one bit is not 1.

$$P(\overline{E}) = 1 - \frac{1}{1024} = \frac{1023}{1024}$$

□

## 6.2 Dependency and Bayes Theorem

### 6.2.1 Conditional Probability

The probability of event A, given that event B happens is given by:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \tag{10}$$

Therefore, the probability of both A and B occuring can be given by:

$$P(A \cap B) = P(A|B) \times P(B) \tag{11}$$

**Definition 6.2.1.1**: Events can occur independently. The occurrence of one event is not dependent on the occurrence of another if, and only if:

$$P(A \cap B) = P(A) \times P(B) \tag{12}$$

**Definition 6.2.1.2**: $P(A \cap B)$ can be presented in two ways, either $P(A|B) \times P(B)$ or $P(B|A) \times P(A)$. Equating these, we can obtain Bayes' Theorem.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)} \tag{13}$$

This can be rewritten by using variables, where $P(A|B)$ is the posterior, $P(A)$ is the prior, $P(B|A)$ is the likelihood, and $P(B)$ is the evidence.

$$posterior = \frac{likelihood \times prior}{evidence} \tag{14}$$

Suppose that $E_1, E_2, ..., E_n$ are mutually exclusive events such that:

$$\bigcup_{i=1}^{n} E_i = S \tag{15}$$

This means that one of the events **must** occur. Then, in any case, Bayes' Theorem states:

$$P(E_k|A) = \frac{P(E_k) \times P(A|E_k)}{\sum_{j=1}^{n} P(E_j) \times P(A|E_j)} \tag{16}$$

## 6.3 Discrete Probability Distributions

## 6.4 Continuous Probability Distributions

## 6.5 Gaussian Distribution and its Characteristics

## 6.6 Correlations

## 6.7 Central Limit Theorem and Sampling

## 6.8 Hypothesis Testing

## 6.9 Data Types and Visualisation

## 6.10 Basic Data Analysis with Python

# 7 Data Cleaning and Analytics

This section is taught by Dr Igor Razgon. The recommended reading for this topic is:

- Python for Data Analytics (McKinney, 2022).


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

## 7.1 Data Quality and How to Produce High-Quality Data

### 7.1.1 Data Quality

**Definition 7.1.1.1**: Data cleansing is the process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table, or database and refers to identifying incomplete, inaccurate, or irrelevant parts of the data and then replacing, modifying, or deleting the dirty or coarse data.

For data to be considered high-quality, it should meet the following criteria:

- **Definition 7.1.1.2**: Validity: The degree to which the measures conform to defined business rules or constraints.
- **Definition 7.1.1.3**: Accuracy: The degree of conformity of a measure to a standard or a true value.
- **Definition 7.1.1.4**: Completeness: The degree to which all required measures are known.
- **Definition 7.1.1.5**: Consistency: The degree to which the data is consistent, within the same data set or across multiple data sets.
- **Definition 7.1.1.6**: Uniformity: The degree to which the data is specified using the same unit of measure.

### 7.1.2 How to Produce High-Quality Data

Producing high-quality data is an iterative process moving through inspection, cleaning, verifying, and reporting.

**Definition 7.1.2.1**: Inspection: Detect unexpected, incorrect, and inconsistent data.

- **Definition 7.1.2.2**: Data profiling is the process of viewing the summary statistics of your data to identify standards or patterns in columns of data. Here it is also important to note the data type, any missing values, the number of unique values, the distribution of values, and the relationship between the data and other columns.
- Visualising the data can also be helpful for identifying outliers, as can using statistical methods such as mean, standard deviation, range, and/or quartiles.

**Definition 7.1.2.3**: Cleaning: Fix or remove the anomalies detected.

- It is important to identify and remove duplicate entries in your data sets.
- When converting the data into a consistent data type, values unable to fit the format should be flagged and converted to NA value.
- Beyond the data types, the data should be further standardised. This means strings are of a consistent capitalisation, numerical values use the same measurement unit, and dates are in the same format (pick either the UK or US format).
- It can also be useful to scale/transform your data to fit within a certain scale (i.e. 0-100 or 0-1). This can make data easier to plot (e.g., exponential data being logged).
- You can either replace missing values by using a mask to indicate where they are missing, or choosing a sentinel value to replace the missing entries. Conversely, you can also drop the whole record from the data set, or where possible calculate the missing values from other observations.
- **Definition 7.1.2.4**: We define outliers as values which fall 1.5 times the interquartile range from both the first and third quartiles.

**Definition 7.1.2.5**: Verifying: After cleaning, the results are inspected to verify correctness.

**Definition 7.1.2.6**: Reporting: A report about the changes made and the quality of the currently stored data is recorded.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 7.2 Working with Files

There is no new content here, it is just an introduction to Python's file read/write operations.

### 7.3 Pandas and Various Features of NumPy

### 7.4 Data Cleaning and Preparation

### 7.5 Python Data Visualisation Libraries

### 7.6 Data Analytics and Reporting

## 8 Computer Graphics

This section is taught by Dr George Alex Koulieris. Check for recommended reading.

## 9 Image Processing

This section is taught by Dr George Alex Koulieris. Check for recommended reading.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part III
# Networks and Systems

## 10 Module Introduction

Networks and Systems has the module code COMP2211 and is led by Dr Wanqing Tu.

### 10.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 6. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Cyber Security | 9th Oct | 5th Nov | 3rd Dec | Dr Carlton Shepherd | 25% |
| Networks | 23rd Oct | 21st Jan | 18th Feb | Dr Wanqing Tu | 25% |
| Dist. Sys. & Databases | May Exam | - | - | Dr Vasantam | 50% |

Table 6: Assessments for Networks and Systems.

### 10.2 Timetable

This module is taught through one two-hour lecture and one two-hour computer class each week as seen below in Table 7.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Thursday | 09:00 - 11:00 | Lecture | CLC202 | Dr Carlton Shepherd |
| Thursday | 16:00 - 18:00 | Computer Class | CC-0007 | - |

Table 7: Michaelmas Term Timetable for Networks and Systems.

## 11 Cyber Security

This section is taught by Dr Carlton Shepherd. The recommended reading for this module is as follows:

- Practical Cryptography for Developers (Nakov, Stefanov and Shideroff, 2018), specifically the section on "Encryption: Symmetric and Asymmetric".

### 11.1 Introduction, Threat Modelling, and Applied Cryptography

#### 11.1.1 Basic Definitions

**Definition 11.1.1.1**: A system's security depends on the definition of the adversary: who they are, what they can do (or their capabilities) and their determination (time, money, resources). This is the adversary or threat model.

**Definition 11.1.1.2**: An asset is something that is worth protecting.

**Definition 11.1.1.3**: A threat is a person or program that may try and access/steal your assets.

**Definition 11.1.1.4**: An assumption is something you are relying on to be true/secure by default.

**Definition 11.1.1.5**: Your goal should be in terms of what needs to be secure, and how long for.

**Definition 11.1.1.6**: The controls are the things you can do to attempt to protect your assets.

The CIA triad are the three main security properties, these are:


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

- **Definition 11.1.1.7**: Confidentiality: The outcomes of different actions in the system are only visible by authorised subjects.
- **Definition 11.1.1.8**: Integrity: Actions modifying the system can only be performed by authorised subjects.
- **Definition 11.1.1.9**: Availability: Subjects can perform all actions they are authorised for.

In addition to these properties, it is also important to consider additional properties, such as:

- **Definition 11.1.1.10**: Authenticity: An entity is who they claim they claim to be.
- **Definition 11.1.1.11**: Non-Repudiation: An actor cannot easily deny that an action has occurred.
- **Definition 11.1.1.12**: Context-Binding: An action is valid only for a particular period of time, distance, location, and/or environment.

**Definition 11.1.1.13**: An exploit is a piece of code, a tool, or a technique that takes advantage of a flaw in software, hardware, or a network to cause unintended behaviour.

**Definition 11.1.1.14**: A vulnerability is a weakness or flaw in a system's design, implementation, operation, or human behaviour that an attacker can exploit to compromise security.

**Definition 11.1.1.15**: Risk is the probability or liklihood that a threat will exploit a vulnerability and cause harm, loss, or negative consequences to an asset.

### 11.1.2 STRIDE Modelling

**Definition 11.1.2.1**: One method of classifying security risks is STRIDE, which was first developed by Microsoft. It is separated as follows:

- **Definition 11.1.2.1**: **S**poofing is the act of using someone else's credentials to perform an operation and is a failure of identification and authentication.
- **Definition 11.1.2.2**: **T**ampering is the malicious modification of data and is a failure of integrity and authenticity.
- **Definition 11.1.2.3**: **R**epudiation is the act of denying having performed an operation and is a failure of accountability and identification.
- **Definition 11.1.2.4**: **I**nformation Dislosure is the exposure of information to unauthorised users and is a failure of confidentiality.
- **Definition 11.1.2.5**: **D**enial of Service is the act of denying service to legitimate users and is a failure of availability.
- **Definition 11.1.2.6**: **E**levation of Privilege is an instance of an unprivileged user gaining privileged access and is a failure of confinement and authentication.

This is a helpful model for structuring the analysis of potential system threats but it is not exhaustive or exclusive.

### 11.1.3 Applied Cryptography

**Definition 11.1.3.1**: To encode some given data, it is transformed from one format/domain to another. This is fully reversible, and does not provide any secrecy. The inverse procedure is called decoding.

- **Definition 11.1.3.2**: A subtype of encoding is steganography, where the existence of the secret message is concealed.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Definition 11.1.3.3**: To hash some given data, it is transformed into a (usually) smaller, fixed-size. This is most commonly considered a one-way function. An example of this are SHA algorithms.

- A hashing algorithm works by taking a message $m$ and applying a hashing algorithm $h = H(m)$. $h$ can then be sent securely and the recipient can then take $h' = H(m')$ where $m = m'$, and if $h' = h$, then the message recieved is correct.

**Definition 11.1.3.4**: Non-secret data (plaintext) can be encrypted into ciphertext using a secret key. The output is $Enc(m, K) = c$. The inverse procedure, decryption, again uses a secret key to transform the ciphertext back to plaintext $Dec(c, K) = m$.

### 11.1.4 Cipher Examples

**Definition 11.1.4.1**: A substitution cipher works by taking each letter in the alphabet and replacing it with another one. This was purportedly created and used by Julius Caesar. These are weak though and can be broken by:

- **Definition 11.1.4.1**: A brute-force attack tries every possible key, which for substitution ciphers becomes trivial.
- **Definition 11.1.4.2**: A frequency analysis attack matches the ciphertext to the known frequency of English letters.

**Definition 11.1.4.4**: Another historical example of a cipher is a Vigenere cipher. This uses a sliding password, such that repeats of the same letter are coded differently. Attacks against this have to guess possible key lengths and then try all possibilities using frequency analysis. This is much harder to break.

**Definition 11.1.4.5**: Another encryption method is using one time pads. Here, you select a key as long as your message. Data is converted into binary and bitwise XORed with the key. To decode it, you then bitwise XOR the ciphertext with the key. This is considered theoretically secure as long as the key generation process is truly random, the key is never reused, and the key is kept secret.

**Definition 11.1.4.6**: Stream ciphers use a short secret key and a nonce - a use-once but public value - to generate a long pseudorandom sequence called a keystream. Only the short secret key must be kept private. We then XOR with the message and keystream.

- To encrypt, you take the secret key $K$ and the nonce $N$ and feed them through a keystream generator. The resulting keystream $S$ is then XORed with the plaintext $p$ to produce ciphertext $C$.
- To decrypt, the recipient takes the same secret key $K$ and nonce $N$, feeds them through the keystream generator and then XOR the result with the ciphertext $C$ to produce the original plaintext $p$.

**Definition 11.1.4.7**: Block ciphers transform a fixed-size block of plaintext into a ciphertext block of the same size, using a secret key. Encryption applies a keyed transformation to the plaintext block, and decryption reverses the transformation using the same key. An example of this is AES encryption. To encrypt longer messages, we use a mode of operation which specifies how to encrypt message blocks in sequence - this usually involves a nonce (initialisation vector).

- **Definition 11.1.4.7**: AES Electronic Code Book (ECB) mode works by taking the plaintext, splitting it into blocks, and then encrypting each of them separately. This is not secure.
- **Definition 11.1.4.8**: AES Cipher Block Chaining (CBC) mode splits the plaintext into blocks and encrypts the first one. The result is then XORed with the second plaintext block before it is encrypted, and so on.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 11.1.5 Public Key Cryptography

**Definition 11.1.5.1**: Rivest, Shamir, Adelman (RSA) is an algorithm where person A computes the product of two prime numbers $n = p \times q$ and makes the value of $n$ public. Person A then generates a public value $e$ and a private value $d$ using $p$ and $q$. $e$ can then be used to encrypt messages to send to person A, and $d$ used to decrypt them. This is secure based on the computational difficulty of factorisation, but with the rise of quantum computing, is becoming less secure.

**Definition 11.1.5.2**: Digital signatures enables others to check that a message was signed using a private key and has not been altered. To sign a message, the sender uses their private key, and then sends the message and signature to the recipient. To verify the signature, the recipient uses the sender's public key and compares the result to the message.

## 11.2 System Security

### 11.2.1 Analysing the Security of Systems

**Definition 11.2.1.1**: A system is a set of components that interact to achieve a purpose. It generally consists of five components:

- **Definition 11.2.1.2**: Subjects are active entities with an identity and capabilities.
- **Definition 11.2.1.3**: Resources are passive entities with an identity and properties.
- **Definition 11.2.1.4**: Actions are interactions between subjects and resources.
- **Definition 11.2.1.5**: The purpose of a system is what it is used for, and so what security must preserve.
- **Definition 11.2.1.6**: A boundary defines what is inside (what you control), and what is outside.

When considering the security of a system, we should consider four questions:

- **What matters?** An asset and a security property (e.g., confidentiality).
- **Who could cause harm?** An adversary with stated capabilities.
- **Why could they succeed?** A weakness and the assumptions it violates.
- **What changes the outcome?** Controls and countermeasures. What is the remaining risk?

**Definition 11.2.1.7**: A trust boundary is where data or control passes between parties that trust each other differently (e.g., a phone to a ticketing service).

**Definition 11.2.1.8**: An attack surface is every entry point an adversary can reach.

**Definition 11.2.1.9**: A trusted computing base (TCB) is what must work correctly for the goal to hold. If any part fails, security fails.

**Definition 11.2.1.10**: A reference monitor sits between the subjects and objects. It recieves requests for the objects, compares them with the security policy, and if it passes, returns the requested objects. It is important when designing a system that this is always invoked, it is tamper-proof, and that the check is verifiable.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

| Actor | Motivation | Typical Capability | Example |
|---|---|---|---|
| Opportunists | Curiosity, notoriety, easy money. | Low: Off-the-shelf tools and mass scanning. | Mirai Botnet (2016). |
| Hacktivists | Publicity for a cause. | Low-to-Medium: DDoS, leaks, defacement. | Operation Payback (2010). |
| Organised Criminals | Money. | Medium-to-High: Ransomware, affiliates, access brokers. | Rhysida (2023). |
| Insiders | Grievance money, or simple mistakes. | High: They already have legitimate access. | Morrisons (2014). |
| Nation States | Espionage, sabotage, strategic advantage. | Very High: Custom tools, patience, zero-days. | SolarWinds (2020). |

Table 8: Potential adversaries to a system.

**Example 11.2.1.1**: The Hezbollah pager attack was an attack where Hezbollah ordered thousands of new pagers for members across Lebanon and Syria. This consignment was intercepted by agents in a supply chain attack and 10 to 20 grams of military grade explosive was packed into each device. These pagers were then delivered to Hezbollah and handed out to members. The pagers then recieved a code, buzzed, and exploded in pockets, on belts, or in faces of users.

**Definition 11.2.1.11**: It can also be useful to consider system security from an adversarial perspective. To do this, people commonly use the kill chain model.

- Reconnaissance: Harvesting email addresses, conference information, etc.
- Weaponisation: Coupling an exploit with a backdoor into a deliverable payload.
- Delivery: Delivering a weaponised bundle to the victim via email, USB, etc.
- Exploitation: Exploiting a vulnerability to execute code on the victim's systems.
- Installation: Installing malware on an asset.
- Command and Control (C2): Setting up a control channel for remote manipulation of victims.
- Actions on Objectives: With "hands on keyboard" access, intruders can accomplish their original goals.

**Definition 11.2.1.12**: Reconnaissance can be split into a few types: passive gathering where attackers have no interaction with the target, active gathering where they do, and open-source intelligence (OSINT) where the attacker analyses data from publically available sources. They do this to identify potential attack vectors.

**Example 11.2.1.2**: The British Library attack in 2023 can be split into the categories shown in the Kill chain:

- Reconnaissance: Hostile reconnaissance on the 25th-26th October was detected and blocked, but treated as a one-off.
- Delivery and Exploitation: Most likely privileged credentials obtained by phishing or brute force, then used on a remote-access server without MFA. No software exploit was needed.
- Installation and C2: The attackers hijacked the network's own administration tools, and deleted log files to cover their tracks.
- Actions on Objectives: A "historically complex" legacy network let them reach well beyond the entry point and roughly 600 GB was copied. The servers were then encrypted or destroyed.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 11.3 Defence

**Definition 11.3.0.1**: Asymmetry in security is the principle that finding a hole is much easier than securing all possible holes. However, an attack is a sequence of stages that are all opportunities to detect or stop it.

**Definition 11.3.0.2**: An attack tree is constructed such that: the root is the attackers' goal, the children are the different ways to achieve it, and the leaves are concrete actions. An OR node indicates that any of the children achieves the parent, while an AND node indicates that all children are required to achieve the parent. A set of leaves that satisfies the root is a complete attack.

When designing a system, it is important to consider the following principles: minimise the attack surface, split your defence across many layers, assign users as few privileges as necessary, deny requests as a default reponse unless explicitly allowed, attackers will always take the weakest link, and you should always assume there will be a breach, so monitor and plan for recovery.

## 12 Networks

This section is taught by Dr Wanqing Tu. Check for recommended reading.

## 13 Databases

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

### 13.1 Enhanced Entity-Relationship Model

### 13.2 Semistructured Databases (XML)

### 13.3 XML Data Manipulation (XPath and XQuery)

### 13.4 Transactions

### 13.5 Concurrency Control

### 13.6 Distributed Transactions

### 13.7 Distributed Concurrency Control

## 14 Distributed Systems

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**14.1 Introduction to Distributed Systems**

**14.2 Middleware and RMI Technologies**

**14.3 Message-Oriented Middleware and Web Services**

**14.4 Replication Models**

**14.5 Concurrent Events and Consistency Control**

**14.6 Fault Tolerance**

**14.7 Load Distribution**


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part IV
# Programming Paradigms

## 15 Module Introduction

Programming Paradigms has the module code COMP2221 and is led by Dr Stuart James.

### 15.1 Assessment

This module is assessed through one piece of summative coursework and two final exams, these can be seen in Table 9. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| *Systems Programming* | *12th Oct* | *12th Nov* | *10th Dec* | *Dr Stuart James* | - |
| Systems Programming | 23rd Oct | 10th Dec | 21st Jan | Dr Stuart James | 50% |
| Object Oriented & Functional | May Exam | - | - | Ivkovic & Moses | 50% |

Table 9: Assessments for Programming Paradigms.

### 15.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 10.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Monday | 11:00 - 12:00 | Lecture | CLC202 | Dr Stuart James |
| Monday | 16:00 - 18:00 | Computer Class | CB-LG001 | n/a |
| Friday | 14:00 - 15:00 | Lecture | CLC202 | Dr Stuart James |

Table 10: Michaelmas Term Timetable for Programming Paradigms.

## 16 Systems Programming

This section is taught by Dr Stuart James. The reading list for this module is as follows:

- Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.

**Definition 16.0.0.1**: Systems programming involves the development of individual pieces of software that allow the entire system to function as a single unit.

### 16.1 Version Control and UNIX

#### 16.1.1 Introduction to UNIX

Files are stored in a hierarchical structure, allowing for grouping. To navigate around these files, we can use UNIX navigation commands as seen in Table 11.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

| Action | Command |
|---|---|
| List the contents of the current folder. | *ls* |
| Change folder/directory. | *cd* |
| Make new folder/directory. | *mkdir* |
| Move a file/folder. | *mv* |
| Copy a file/folder. | *cp* |
| Delete a file or, by adding $-r$, a directory. | *rm* |
| Check file/folder size. | *du* |
| List all files. | *find* |

Table 11: UNIX file navigation commands.

## 16.2 The Basics of Shell

All shell files are saved with a *.sh* extension and must begin with #!*/bin/bash*.

### 16.2.1 Basic Commands

```
#!/bin/bash
for f in *
do
    #something in here
    echo $f
done
```

Algorithm 1: For loop example.

```
./myscript.sh 10 2

#!/bin/bash
if [$1 -lt $2]
then
    echo "yes" $1 "is less than" $2
else
    echo "no it isn't"
fi
```

Algorithm 2: If statement example (compares 10 and 2).

## 16.3 The Basics of C

### 16.3.1 Basic File Setup

**Definition 16.3.1.1**: Pre-processor directives are lines which start with a # and are commands to the C pre-processor.

- If the directive is #include, and <> are used, then */usr/include* is prioritised in UNIX by convention, whereas if "" are used, then the current directory is prioritised.
- **Definition 16.3.1.1**: If the directive is #define, then this is a definition macro. These are used to provide definitions in code.

- **Definition 16.3.1.2**: You can also use directives for debugging without generating overheads, these are called conditionals. They are as shown in Table 12.

| Conditional | Function |
|---|---|
| #ifdef VAR | Tests if VAR is #defined. |
| #ifndef VAR | Tests if VAR is not #defined. |
| #else | Runs if the original condition evaluates as false. |
| #endif | Ends the if testing condition. |

Table 12: Different types of conditionals.

You can also put debugging code in the preamble, as seen in Algorithm 3.

```
#define MY_DEBUG // Define an identifier.

#ifdef MY_DEBUG
    assert(i > 0);
    printf("i is %d \n", i);
#endif
```

Algorithm 3: Debugging Code Example.

**Definition 16.3.1.4**: You can also use #define to define parameterised macros. These act like functions, and an example can be seen in Algorithm 4.

```
#define ADD(a,b) a + b
```

Algorithm 4: Parameterised Macro Example.

**Definition 16.3.1.5**: The main() function is the entry function for C programs. You can only have one of these.

### 16.3.2 Printf Parameters

| Value | Resulting Format |
|---|---|
| $\%d$ | Signed decimal (int). |
| $\%u$ | Unsigned decimal. |
| $\%o, \%x$ | Octal and hexidecimal. |
| $\%l$ | Long. |
| $\%f$ | Floating point, so $\%1.2f$ will give 3.14. |
| $\%e$ | Floating point (exponent form). |
| $\%c, \%s$ | Character and string. |

Table 13: First parameter of printf.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 16.4 Control Flow and Functions
### 16.5 Data Types and Structures
### 16.6 Advanced Bash Scripting
### 16.7 Pointers
### 16.8 Memory Management
### 16.9 Debugging
### 16.10 Scope and Recursion
### 16.11 Libraries
### 16.12 stdlib
### 16.13 C++

## 17 Functional Programming

This section is taught by Dr Billy Moses Jr.. Check for recommended reading.

## 18 Object Oriented Programming

This section is taught by Dr Mladen Ivkovic. Check for recommended reading.

### 18.1 Java Fundamentals
### 18.2 OOP with Classes and Objects
### 18.3 Encapsulation
### 18.4 Inheritance
### 18.5 Abstraction with Abstract Classes
### 18.6 Abstraction with Interfaces
### 18.7 Polymorphism
### 18.8 Java Collections
### 18.9 Best Practises


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part V
# Software Engineering

## 19 Module Introduction

Software Engineering has the module code COMP2281 and is led by Dr Craig Stewart.

### 19.1 Assessment

This module is assessed through a group project, the deadlines for this can be seen in Table 14. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Peer Evaluation 1 | 16th Oct | 19th Nov | 10th Dec | Dr Craig Stewart | 1.67% |
| Requirement Documentation | 16th Oct | 19th Nov | 17th Dec | Dr Craig Stewart | 15% |
| Peer Evaluation 2 | 16th Oct | 11th Feb | 11th Mar | Dr Craig Stewart | 1.67% |
| Product Presentation | Week 16 | - | 18th Mar | Dr Craig Stewart | 15% |
| Technical Report | 5th Feb | 18th Mar | 6th May | Dr Craig Stewart | 30% |
| Peer Evaluation 3 | 5th Feb | 18th Mar | 6th May | Dr Craig Stewart | 1.67% |
| Git Commit | 5th Feb | 29th Apr | 1st Jul | Dr Craig Stewart | 5% |
| Project Handover | 5th Feb | 29th Apr | 1st Jul | Dr Craig Stewart | 10% |
| Reflective Experience Report | 5th Feb | 6th May | 1st Jul | Dr Craig Stewart | 20% |

Table 14: Assessments for Software Engineering.

### 19.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 15.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Tuesday | 09:00 - 10:00 | Lecture | TLC033 | Dr Nelly Bencomo |
| Wednesday | 11:00 - 12:00 | Lecture | TLC033 | Dr Nelly Bencomo |
| Thursday | 11:00 - 13:00 | Practical | CC-0007 | Dr Gagangeet Aujla |

Table 15: Michaelmas Term Timetable for Software Engineering.

These lectures are taught jointly by Dr Craig Stewart and Dr Nelly Bencomo.

## 20 Project Management

### 20.1 Your Team, Formation, and Leadership

This lecture was taught by Dr Nelly Bencomo.

Add notes for this lecture.

## 21 Software Engineering

## 22 Legal, Social, Ethical, and Professional Issues

This recommended reading for this sub-topic is:


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

- The Software Engineering Code of Ethics and Professional Practice, ACM (2024).

## 22.1 Introduction to Software Engineering and Software Engineering Ethics

This lecture was taught by Dr Craig Stewart.

### 22.1.1 Introduction to Software Engineering

**Definition 22.1.1.1**: An ill-structured problem, as defined by (Simon, 1973), is: elusive, unbounded, not understandable, without end conditions. They require orientation and context, and have many stakeholders and many possible solutions.

**Definition 22.1.1.2**: A good problem solver needs the following:

- **Method**: Refers to a formal procedure for accomplishing a goal, typically independent of the tools used.
- **Tool**: An instrument or automated system for accomplishing something in a better way.
- **Procedure**: A combination of tools and techniques to produce a product.
- **Paradigm**: The philosophy or approach for building a project.

**Definition 22.1.1.3**: For this module, a stakeholder will generally fall into one of three roles:

- **Customer/Client**: The company, organisation, or person who pays for the software system.
- **Developer**: The company, organisation, or person who is building the software system.
- **User**: The person/people who will actually use the system.

### 22.1.2 Introduction to Ethics

**Definition 22.1.2.1**: The software engineering code of ethics is defined by Rost and Glass (2010) and summarised more thoroughly by the Association for Computing Machinery (2024).

# 23 Group Project


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part VI
# Theory of Computation

## 24 Module Introduction

Theory of Computation has the module code COMP2181 and is led by Dr Barnaby Martin.

### 24.1 Assessment

This module is assessed through two summative benchtests, and one final exams, these can be seen in Table 16. Additional formatives are also shown in italics.

| Title | Opening | Due | Feedback | Professor | Weight |
|---|---|---|---|---|---|
| Benchtest 1 | Week 11 | - | 2nd Feb | Dr Dantchev and Dr Mertzios | 12.5% |
| Benchtest 2 | Week 20 | - | - | Dr Dantchev and Dr Mertzios | 12.5% |
| Final Exam | May Exam | - | - | Dr Barnaby Martin | 75% |

Table 16: Assessments for Theory of Computation.

### 24.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 17.

| Day | Time | Class Type | Location | Lecturer |
|---|---|---|---|---|
| Tuesday | 10:00 - 11:00 | Lecture | CLC202 | Dr Barnaby Martin |
| Wednesday | 09:00 - 11:00 | Problem Class | TLC116 | - |
| Thursday | 14:00 - 15:00 | Lecture | CLC202 | Dr Sukanya Pandey |

Table 17: Michaelmas Term Timetable for Theory of Computation.

## 25 Models of Computation

This section is taught by Dr Barnaby Martin. Check for recommended reading.

### 25.1 Finite-State Automata and Regular Languages

#### 25.1.1 Formal Definitions

**Definition 25.1.1.1**: A deterministic finite-state automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$ where $Q$ is the finite set of states, $\Sigma$ is a finite alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

**Definition 25.1.1.2**: A DFA $M$ recognises the language $L$ if $L = \{w | M \text{ accepts } W\}$.

**Definition 25.1.1.3**: A language is called a regular language if some DFA recognises it.

**Definition 25.1.1.4**: A regular expression $R$ defines a language $L(R)$. We shall eventually prove that $RE \equiv DFA$. $R$ is a regular expression over the alphabet $\Sigma$ if $R$ is:

1. $a$ for some $a \in \Sigma$.
2. $\epsilon$ (the empty string).
3. $\emptyset$ (the empty set).


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

4. $(R_1 \cup R_2)$ where $R_1$ and $R_2$ are regular expressions.
5. $R_1 \cdot R_2$ where $R_1$ and $R_2$ are regular expressions.
6. $R_1^*$ where $R_1$ is a regular expression.

The list of regular operations can be seen in Table 18 where the operations labelled with $^+$ are language theory specific.

| Operation Name | Notation | Set Equivalent |
|---|---|---|
| Union | $A \cup B$ | $\{x \mid x \in A \text{ or } x \in B\}$ |
| Intersection | $A \cap B$ | $\{x \mid x \in A \text{ and } x \in B\}$ |
| Difference | $A \backslash B$ | $\{x \mid x \in A \text{ and } x \notin B\}$ |
| Complement | $\overline{A}$ | $\Sigma * \backslash A$ |
| Concatenation$^+$ | $A \circ B$ | $\{xy \mid x \in A \text{ and } y \in B\}$ |
| Star$^+$ | $A*$ | $\{x_1x_2...x_k \mid k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\} \equiv \cup_{i=0}^{\infty} A^i$ |

Table 18: Regular Operations.

### 25.1.2 Combining Automata

Sometimes we may want to implement an operation on multiple languages. We can do this by combining their automata. If we have that $L_1$ is recognised by $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $L_2$ is recognised by $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$; we want to combine $M_1$ and $M_2$ into a new automaton $M$ that would recognise $L_1 \cup L_2$.

You would think that we could just run the automata sequentially, and accept if either $M_1$ or $M_2$ accept, but this does not work. This is because after $M_1$ has run on the input, the input is exhausted and there is no way to rewind it in order to run $M_2$.

Instead, we can attempt to run $M_1$ and $M_2$ on the input in parallel. We do this by using pairs of states (one for each DFA). Formally, the components of the new automaton $M = (Q, \Sigma, \delta, q_0, F)$ are:

1. The set of possible states contains every possible cross-set pair of $Q_1$ and $Q_2$.

$$Q = Q_1 \times Q_2$$

2. For every $s \in Q_1, u \in Q_2, a \in \Sigma$. The transition function contains every transition from a pair of states from $Q$ being given any input from $\Sigma$.

$$\delta((s, u), a) = (\delta_1(s, a), \delta(u, a))$$

3. The start state is the pair containing both start states from $M_1$ and $M_2$.

$$q_0 = (q_1, q_2)$$

4. The set of accept states is every possible cross-set pair of accept states from $F_1$ and $F_2$.

$$F = \{(s, u) \in Q \mid s \in F_1 \text{ or } u \in F_2\}$$

### 25.1.3 Non-Deterministic Finite Automaton

If, instead you wish to work with concatenation, such that $w \in L_1 \circ L_2$ only if there are words $x$ and $y$ such that $w = xy$, $x \in L_1$ and $y \in L_2$. You need to run $M_1$ on a prefix of $w$ and then run $M_2$ on the rest. This must be done non-deterministically and stops becoming a DFA as all accept states of $M_1$ have to map to the start state of $M_2$.

**Definition 25.1.3.1**: A non-deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where $Q$ is the finite set of states, $\Sigma$ is a finite alphabet, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathscr{P}(Q)$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Computing NFA**

Before computing an NFA, an important convention to note is that a single step in a computation consists of first reading the next input symbol followed by taking any number of $\epsilon$-transitions. This means that empty strings $\epsilon$s can be freely added inside the actual word $\in \Sigma*$, e.g., 010110 can be viewed as, say, $\epsilon 01\epsilon\epsilon 01\epsilon 01$.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA and $w = w_1 w_2 ... w_n$ be a word over $\Sigma \cup \{\epsilon\}$. $M$ accepts $w$ if there exists a sequence of states $r_0, r_1, r_2, ..., r_n$ satisfying the following conditions:

1. $r_0 = q_0$.
2. $r_{i+1} \in \delta(r_i, w_{i+1})$ for every $i, 0 \leq i \leq n-1$.
3. $r_n \in F$.

### 25.1.4 Converting NFA into DFA

The difference between a DFA and NFA is that a DFA has a unique computation (path) and either accepts or rejects, while an NFA generally has many computation paths and accepts if at least one computation accepts and only rejects if all the computations reject.

**Theorem 25.1.4.1**: Every NFA has an equivalent DFA.

*Proof*: To formally convert an NFA to a DFA, let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA recognising some language $L$ (assume that $N$ has no $\epsilon$-transitions). We shall construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ that recognises the same language $L$ as follows.

1. $Q' = \mathscr{P}(Q)$.
2. For $R \in Q'$ and $a \in \Sigma$, let

$$\delta'(R, a) = \{q \in Q | q \in \delta(r, a) \text{ for some } r \in R\} \tag{17}$$

   equivalently,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a) \tag{18}$$

3. $q_0' = \{q_0\}$.
4. $F' = \{R \in Q' | \exists r \in R, r \in F\}$.

Now, to consider $\epsilon$-transitions, for $R \subseteq Q$, let

$$E(R) = \{q | q \text{ is reachable from } R \text{ through a number of } \epsilon\text{-transitions.}\} \tag{19}$$

Now, taking the above steps, we modify (17) to

$$\delta'(R, a) = \{q \in Q | q \in E(\delta(r, a)) \text{ for some } r \in R\} \tag{20}$$

and (18) to

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)) \tag{21}$$

We then modify step 3 to say $q_0' = E(\{q_0\})$.

□

**Corollary 25.1.4.1**: A language is regular iff some NFA recognises it.

*Proof*: See Theorem 25.1.4.1.

□


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

**Theorem 25.1.4.2**: A language is regular iff some regular expression describes it.

*Proof*: $\Leftarrow$ Given a regular expression $R$, construct a NFA that recognises $L(R)$ by structural induction:

- *Base Case*: All initial regular expressions can be implemented by NFAs.
- *Inductive Step*: We have already proven that the class of regular languages is closed under regular operations.

$\Rightarrow$ Given a DFA $M$, construct a regular expression that recognises $L(M)$.

1. Convert the given $n$-state DFA into an equivalent $n+2$ state GNFA.
2. Eliminate all the internal states of the GNFA one-by-one, while preserving the language recognised by the automaton.
   (a) Pick an internal state $r$ for elimination.
   (b) For every two states $u$ and $v$ with transitions between them and $r$, $u \neq r, v \neq r$, combine the regular expressions of the three into one, new expression.
3. $L(R)$ should now equal $L(M)$.

□

**Definition 25.1.4.1**: A generalised non-deterministic finite automaton is defined as having a single start state with no incoming edges, a single end state with no outgoing edges, and every transition being labelled with a regular expression.

### 25.1.5 Non-Regular Expression

**Definition 25.1.5.1**: A non-regular expression is a language which cannot be recognised by any DFA. We prove a language is non-regular by assuming there is a DFA that recognises language $L$ and then use an adversary argument against that DFA to get a contradiction.

**Lemma 25.1.5.1**: The pumping lemma states that, for every regular language $L$, there is a number $p$ (the pumping length of $L$) such that every word $w \in L$ of length at least $p$ can be divided into three parts $w = xyz$, satisfying the following conditions:

1. $xy^iz \in L$ for every $i \in N$ (in particular $xz \in L$ as one can take $i = 0$).
2. $y$ is a non-empty string.
3. The length of the string $xy$ is not greater than $p$.

*Proof*: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA that recognises $L$, and let us set the pumping length $p$ to be the number of states in $M : p = |Q|$. Consider the computation of $M$ on the word $w = w_1w_2...w_n$ where $n \geq p$.

$$q_0 = r_0 \rightarrow w_1 \rightarrow r_1 \rightarrow w_2 \rightarrow ...r_{n-1} \rightarrow w_n \rightarrow r_n \in F \tag{22}$$

Let $i$ be the first moment in time when we see a repetition in the sequence of states $r_0, r_1, ..., r_n$ (such an $i$ must exist by the Pigeon-Hole principle, as $n+1 > p$). i.e., $r_i = r_j$ for some $i > j$ and the states $r_0, r_1, ..., r_{i-1}$ are all different. We can then set

$$x = w_1w_2...w_j \quad y = w_{j+1}...w_i \quad z = w_{i+1}...w_n \tag{23}$$

It is now straightforward to check that all three conditions are met.

□

Reached slide 41/55.

### 25.2 Büchi Automata and Omega-Regular Languages

### 25.3 Linear-Time Logic and Translating Into Büchi Automata

# 26 Algorithms and Complexity

This section is taught by Dr Sukanya Pandey. The recommended reading for this topic is:

- Introduction to Algorithms (Cormen T.H., et al., 2009).

## 26.1 Asymptotic Complexity and Notation

Lecture one was just a recall of some of last year's Algorithms and Data Structures module.

## 26.2 Greedy Algorithms

**Definition 26.2.0.1**: The knapsack problem is defined as one of two variants. Either:

- The 0-1 knapsack problem (the classic variant), where a thief robs a store and finds $n$ items. Item $i$ is worth $v_i$ euros and weighs $w_i$ kilos. The thief can carry at most $W$ kilos in his knapsack. *Which items can he take to maximise his profit?*
- the fractional knap problem, which has the same setup, but this time the thief can take fractions of items.
- We can assume (without loss of generality) that the knapsack is not big enough to carry all items:

$$W < w_1 + w_2 + ... + w_n \tag{24}$$

### 26.2.1 The Greedy Approach

To solve the fractional problem, the greedy approach would be to calculate the value per kilo using (25) for each item. You then sort the items by value per kilo, in decreasing order. You then take as much of the first item as possible, then the second item, etc., until you reach the capacity of the knapsack $W$.

$$r_i = \frac{v_i}{w_i} \tag{25}$$

**Example 26.2.1.1**: For example, if $W = 10$, and $\mathbf{v} = \{4, 10, 12, 20\}$ and $\mathbf{w} = \{4, 2, 8, 5\}$. We can then calculate $\mathbf{r} = \{1, 5, 1.5, 4\}$. The greedy algorithm then dictates that you should take all of item 2, all of item 4, and 3 kilos of item 3 $W = 2 + 5 + 3$. This gives a total value of $V = 10 + 20 + (3 \times 1.5) = 34.5$.

The running time of this algorithm can be calculated by taking that:

1. Step 1, calculating the value per kilo takes $\Theta(n)$.
2. Step 2, sorting the value per kilo array takes $\Theta(n \; log \; n)$.
3. Step 3, selecting the items can be done in $\Theta(n)$.

Therefore, the worst-case running time is $\Theta(n \; log \; n)$.

However, approach will not work for the 0-1 knapsack problem.


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

### 26.2.2 Correctness of the Greedy Algorithm for Fractional Knapsack

$n$ items are sorted by value per kilo, (25) in decreasing order, i.e.,

$$r_1 \geq r_2 \geq ... \geq r_n \tag{26}$$

Two items, $p$ and $q$ with $r_p = r_q$ are considered to be equivalent. Hence, we may assume that

$$r_1 > r_2 > ... > r_n \tag{27}$$

The total available amount of item $i$ is $w_i$. Let $k$ be such that

$$w_1 + w_2 + ... + w_k \leq W \tag{28}$$

and

$$w_1 + w_2 + ... + w_{k+1} > W \tag{29}$$

Let $\mathbf{x} = (x_1, x_2, ..., x_n)$ denote a solution: $0 \leq x_i \leq w_i$ is the amount the thief takes of item $i$ for $i = [0, n]$. The total value taken is then given by

$$\sum_{i=1}^{n} r_i x_i \tag{30}$$

The greedy solution $\mathbf{x}$ is given by $x_i = w_i$ for $1 \leq i \leq k$ and

$$x_{k+1} = W - (w_1 + w_2 + ... + w_k) \tag{31}$$

$x_i = 0$ for $k + 2 \leq i \leq n$. Let $\mathbf{x}^* = (x_1^*, x_2^*, ..., x_n^*)$ be an optimal solution.

Suppose that $\mathbf{x}^* \neq \mathbf{x}$. Then

- $x_i^* < x_i$ for some $i \leq k + 1$.
- Let $j = \min\{i : x_i^* < x_i\}$.
- And $x_l^* > x_l$ for some $l > j$.

Let us add some more of item $j$ to the optimal choice. Let $y$ be another solution, defined by

$$y = \begin{cases} y_j = \min\{w_j, x_j^* + (x_l^* - x_l)\} \\ y_l = x_l^* - (y_j - x_j^*) \\ y_i = x_i^* \text{ otherwise.} \end{cases} \tag{32}$$

## 26.3 Divide and Conquer Algorithms

## 26.4 Dynamic Programming Algorithms

## 26.5 Machinings in Graphs


*LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson*

# Part VII
# Appendix

## 27 Artificial Intelligence

### 27.1 Module Professors

**Module Lead**: Doctor Yang Long

- Focus: Scalable AI solutions for large-scale healthcare applications.
- Email: yang.long@durham.ac.uk
- Office Hours: Monday 15:30 - 16:30 in Term 1.

**Lecturer**: Professor Iain Stewart

- Focus: Computational complexity, graph/group theory, and theoretical aspects artificial intelligence.
- Email: i.a.stewart@durham.ac.uk
- Office Hours: Friday 14:00 - 15:00 in Term 2.

**Lecturer**: Doctor Shauna Concannon

- Focus: Computational linguistics, social interaction, and human-computer interaction.
- Email: shauna.j.concannon@durham.ac.uk
- Office Hours: Thursday 14:30 - 15:30 in Term 2.

### 27.2 References

### 27.3 List of Figures

### 27.4 Reading List

#### 27.4.1 Machine Learning

Long, Y. and Durham University (2026). COMP2261_Machine_Learning_Study_and_Coursework_Guide-_with_Navigation.pdf.

Deisenroth, M.P., Faisal, A.A. and Ong, C.S. (2020). Mathematics for Machine Learning. [online] Cambridge University Press. Available at: https://mml-book.com/ [Accessed 3 Oct. 2026].

Hackeling, G. (2017). Mastering Machine Learning with scikit-learn. 2nd edn. Packt Publishing.

#### 27.4.2 Ethics and Bias in AI

Various, will be added over time.

# 28 Data Science

## 28.1 Module Professors

**Module Lead**: Professor Thomas Erlebach

- Focus: Algorithms and networks.
- Email: thomas.erlebach@durham.ac.uk
- Office Hours: n/a.

**Lecturer**: Doctor Igor Razgon

- Focus: Algorithms and complexity, graph theory, and discrete mathematics.
- Email: igor.razgon@durham.ac.uk
- Office Hours: Friday 14:00 - 15:00.

**Lecturer**: Doctor George Alex Koulieris

- Focus: Imagr processing and computer graphics.
- Email: georgios.a.koulieris@durham.ac.uk
- Office Hours: Please email to make an appointment.

## 28.2 References

## 28.3 List of Figures

## 28.4 Reading List

### 28.4.1 Probability

Degroot, M.H. and Schervish, M.J. (2012). Probability and Statistics. Boston Mass.: Pearson.

### 28.4.2 Data Cleaning and Analytics

McKinney, W. (2022). Python for Data Analysis. O'Reilly Media, Inc.

# 29 Networks and Systems

## 29.1 Module Professors

**Module Lead**: Doctor Wanqing Tu

- Focus: Computer networks and wireless communications.
- Email: wanqing.tu@durham.ac.uk
- Office Hours: Thursday 11:00 - 12:00.

**Lecturer**: Doctor Thirupathaiah Vasantam

- Focus: Applied probability and performance modelling.

- Email: thirupathaiah.vasantam@durham.ac.uk
- Office Hours: Monday 14:00 - 15:00.

**Lecturer**: Doctor Carlton Shepherd

- Focus: Cyber security.
- Email: carlton.g.shepherd@durham.ac.uk
- Office Hours: Email to make an appointment.

## 29.2 References

## 29.3 List of Figures

## 29.4 Reading List

### 29.4.1 Cyber Security

Nakov, S., Stefanov, M. and Shideroff, M. (2018). Practical Cryptography for Developers. [online] sec. Encryption: Symmetric and Asymmetric Available at: https://cryptobook.nakov.com/ [Accessed 9 Oct. 2026].

### 29.4.2 Databases

Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

### 29.4.3 Distributed Systems

Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].

# 30 Programming Paradigms

## 30.1 Module Professors

**Module Lead**: Doctor Stuart James

- Focus: Artificial intelligence, computer vision, human computer interaction, and digital humanities.
- Email: stuart.a.james@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Billy Moses Jr.

- Focus: Algorithms and biologically inspired computing.
- Email: william.k.moses-jr@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Mladen Ivkcovic

- Focus: Performance prediction and optimisation.
- Email: mladen.ivkovic@durham.ac.uk
- Office Hours: Email to make an appointment.

## 30.2 References

## 30.3 List of Figures

## 30.4 Reading List

### 30.4.1 Systems Programming

Bryant, R. and O'Hallaron, D. (2015). Computer systems: A Programmer's Perspective. 3rd edn. Pearson Education, Limited.

# 31 Software Engineering

## 31.1 Module Professors

**Module Lead**: Doctor Craig Stewart

- Focus: Adaptive systems, HCI, and user modelling.
- Email: craig.d.stewart@durham.ac.uk
- Office Hours: Friday 11:00 - 12:00.

**Lecturer**: Doctor Nelly Bencomo

- Focus: Software engineering for autonomous, self-adaptive, AI and ML systems.
- Email: nelly.bencomo@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Gagangeet Aujla

- Focus: Data-driven and data-intensive cyber-physical systems and digital twins.
- Email: gagangeet.s.aujla@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Gokberk Kabacaoglu

- Focus: Machine learning and numerical applications.
- Email: gokberk.kabacaoglu@durham.ac.uk
- Office Hours: Thursday 14:00 - 15:00.

**Lecturer**: Doctor Ryan Crosby

- Focus: The assessment of novice programmers.
- Email: ryan.crosby@durham.ac.uk
- Office Hours: Email to make an appointment.

**Lecturer**: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Email to make an appointment.

## 31.2 References

### 31.2.1 Legal, Social, Ethical, and Professional Issues

Simon, H.A. (1973). The Structure of Ill Structured Problems. Artificial Intelligence, 4(3-4).

Rost, J. and Glass, R.L. (2010). The Dark Side of Software Engineering: Evil on Computing Projects. Washington, DC: IEEE Computer Society.

## 31.3 List of Figures

## 31.4 Reading List

### 31.4.1 Legal, Social, Ethical, and Professional Issues

Association for Computing Machinery (2024). The Software Engineering Code of Ethics and Professional Practice. [online] acm.org. Available at: https://www.acm.org/code-of-ethics/software-engineering-code [Accessed 7 Oct. 2026].

# 32 Theory of Computation

## 32.1 Module Professors

**Module Lead**: Doctor Barnaby Martin

- Focus: Complexity theory, finite model theory, and the links between logic and complexity.
- Email: barnaby.d.martin@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

**Lecturer**: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

**Lecturer**: Doctor Stefan Dantchev

- Focus: Combinatorics.
- Email: s.s.dantchev@durham.ac.uk
- Office Hours: To be confirmed.

**Lecturer**: Doctor George Mertzios

- Focus: Algorithms and networks.
- Email: george.mertzios@durham.ac.uk
- Office Hours: To be confirmed.

## 32.2 References

## 32.3 List of Figures

## 32.4 Recommended Reading

### 32.4.1 Algorithms and Complexity

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). Introduction to Algorithms. 3rd edn. MIT Press.